

Panduan Membuat FULL RBAC schema dan relasi (production-ready) pakai MySQL, Prisma v6 dan Bun JS standar enterprise (5 tabel utama + relasi many-to-many).

Dosen Pengampu : Syahru Rahmayuda, M.Kom Matakuliah : Pemrograman Web Lanjut

Catatan: Karena kita menggunakan DBMS MySQL maka kita mesti menurunkan Prisma ke Versi 6 MODUL 1 Panduan lengkap step-by-step (dari nol sampai jadi) untuk FULL RBAC (5 tabel + relasi many-to-many) menggunakan: • Runtime: Bun • ORM: Prisma v6 • Database: MySQL Kita akan menyusun seperti modul praktikum + production starte

STEP 1 — Persiapan Environment Install Bun, jika kalian belum punya Bun. Berikut ini perintahnya : `curl -fsSL https://bun.sh/install | bash`

Kemudian cek versi bun nya, dengan perintah : `bun --version`

Jika belum ada Install MySQL maka kamu mesti menginstall terlebih dahulu MySQLnya dengan perintah sebagai berikut, jika kamu menggunakan linux Ubuntu, karena kita pada saat praktek menggunakan versi OS linux Ubuntu maka : `sudo apt install mysql-server`

Selanjutnya Login ke MySQL dengan perintah (semua kerjakan di terminal): `mysql -u root -p`

Buatlah database dengan nama `rbac_db` dengan perintah sebagai berikut : `CREATE DATABASE rbac_db;`

STEP 2 — Inisialisasi Project Buat sebuah folder dengan nama `rbac-bun`. Perintah sebagai berikut ini : `mkdir rbac-bun cd rbac-bun`

`bun init -y`

Setelah kita membuat folder project kita buat struktur folder untuk full RBAC Standard Enterprise dan menggunakan Clean Architecture, Berikut adalah struktur yang sudah disesuaikan agar cocok dengan schema.prisma UUID kita dan runtime Bun:

```
rbac-bun/ ├── prisma/ | ├── schema.prisma | └── seed.ts # Untuk mengisi data Role & Permission awal
├── src/ | ├── domain/ # (Core) Business Logic & Types | | ├── entities/ # Definisi objek User, Role
(mirip prisma tapi murni TS) | | └── repositories/ # Interface (Kontrak) untuk database | | | ├──
application/ # (Usecases) Orchestration | | | ├── usecases/ # AssignRoleToUser.ts, CheckPermission.ts | | |
└── services/ # Domain services jika logika melibatkan > 1 entity | | | ├── infrastructure/ # (External
Tools) Detail Teknis | | | ├── database/ # prisma-client.ts (Singleton) | | | └── repositories/ # Implementasi
nyata Prisma (PrismaUserRepository.ts) | | | ├── interfaces/ # (Delivery) Entry Points | | | ├── http/ #
Controllers & Routes (Elysia/Hono) | | | └── middleware/ # RBACMiddleware.ts (Pengecekan Permission) |
| | └── config/ # Env variables & Constants | ├── .env ├── package.json └── bun.lockb
```

Struktur folder yang kita buat diatas adalah implementasi dari Clean Architecture (pola Robert C. Martin). Tujuan utamanya adalah pemisahan kepentingan (Separation of Concerns), di mana aturan bisnis inti tidak bergantung pada framework atau database. Berikut adalah penjelasan detail untuk setiap lapisan:

1. Lapisan domain/ (The Core) Ini adalah lapisan paling dalam dan paling suci. Tidak boleh ada dependensi ke library luar (seperti Prisma atau Elysia) di sini.
 - entities/: Berisi definisi objek bisnis murni dalam bentuk Class atau Type TypeScript. Meskipun mirip dengan model Prisma, entitas ini adalah apa yang dipahami oleh bisnis, bukan database.
 - repositories/: Berisi Interface (kontrak).

Contoh: IUserRepository. Ia hanya mendefinisikan apa yang bisa dilakukan (misal: findById), bukan bagaimana caranya.

2. Lapisan application/ (The Orchestrator) Lapisan ini mengatur alur data dari dan ke lapisan domain.
 - usecases/: Satu file per satu aksi bisnis. Contoh CheckPermission.ts hanya fokus pada logika pengecekan apakah user memiliki akses. Ini menghindari file servis yang raksasa dan sulit dibaca.
 - services/: Digunakan jika ada logika bisnis yang melibatkan banyak entitas sekaligus dan tidak cocok diletakkan di satu usecase.
3. Lapisan infrastructure/ (The Tools) Di sinilah detail teknis berada. Lapisan ini melayani kebutuhan lapisan application.
 - database/: Tempat konfigurasi Prisma Client sebagai singleton agar koneksi ke MySQL efisien di runtime Bun.
 - repositories/: Implementasi nyata dari interface di folder domain. Di sini kita menulis kode Prisma seperti prisma.user.findUnique(). Jika besok kita ganti ke Drizzle ORM, kita hanya perlu mengubah isi folder ini saja.
4. Lapisan interfaces/ (The Delivery) Bagaimana dunia luar berinteraksi dengan aplikasi kita.
 - http/: Tempat kita mendefinisikan framework (Elysia, Hono, atau Express), rute API, dan Controller yang menerima request serta mengirim response.
 - middleware/: Sangat krusial untuk RBAC. RBACMiddleware.ts akan mencegat request, mengambil token, dan memanggil Usecase untuk memvalidasi izin akses sebelum request masuk ke Controller.
5. Folder prisma/ & config/
 - prisma/seed.ts: Sangat penting untuk Enterprise RBAC. Digunakan untuk mengisi data master seperti role ADMIN, USER, dan ratusan Permissions secara otomatis saat pertama kali setup atau migrasi.
 - config/: Sentralisasi konfigurasi seperti JWT Secret, Port Server, atau durasi expiry token agar tidak tersebar (spageti) di banyak file. Berikut ini gambar alur kerja (Data flow projek kita)

Alur Kerja (Data Flow) 1. Request masuk via interfaces/http. 2. Middleware mengecek token dan izin. 3. Controller memanggil Usecase di lapisan application. 4. Usecase berinteraksi dengan Entity dan memanggil Repository. 5. Repository (di infrastructure) mengambil data dari MySQL menggunakan Prisma. 6. Data dikembalikan ke atas hingga menjadi Response JSON.

Mengapa ini "Anti-Spaghetti"? 1. Mudah Diuji (Testable): kita bisa mengetes logika di usecases tanpa perlu menyalakan database. 2. Mudah Diganti: Ingin ganti framework dari Elysia ke Hono? Cukup ubah folder interfaces. Folder domain dan application tidak akan berubah sebaris pun. 3. Skalabilitas: Saat tim bertambah besar, setiap orang bisa fokus di layer tertentu tanpa saling tabrakan kode.

Lanjut penginstalasi dependency nya, di praktek ini kita akan menggunakan dependency prisma versi 6, dengan perintah instalasi sebagai berikut ini :

Instal versi 6 spesifik

```
bun add @prisma/client@^6.0.0 bun add -d prisma@^6.0.0
```

Setelah itu, cek versi untuk memastikan: `bunx prisma -v`

Hasilnya cek versi prisma dapat dilihat pada gambar dibawah ini :

Lanjut kita melakukan Inisiasi Database dengan menjalankan perintah dibawah ini untuk membuat folder konfigurasi Prisma: `bunx prisma init --datasource-provider mysql`

Perintah ini menghasilkan dua file penting: 1. `prisma/schema.prisma`: Tempat kita menulis struktur tabel. 2. `.env`: Tempat menyimpan connection string database.

Langkah berikutnya kita akan melakukan konfigurasi koneksi (`.env`), Buka file `.env`, lalu ganti baris `DATABASE_URL` sesuai dengan settingan MySQL kalian (XAMPP, Docker, atau MySQL Local):

Format:

mysql://USER:PASSWORD@HOST:PORT/NAMA_DATA BASE

```
DATABASE_URL="mysql://root:password_kamu@localhost:3306/rbac_db"
```

Catatan: Buat database kosong bernama `rbac_db` terlebih dahulu di MySQL (bisa via phpMyAdmin atau CLI: `CREATE DATABASE rbac_db;`).

Langkah selanjutnya kita tulis schema RBAC (Standard Enterprise) langkah pertama kita buka `prisma/schema.prisma` dan masukkan skema 5 tabel utama berikut. Skema ini menggunakan relasi Many-to-Many eksplisit agar performa stabil di production.

```
generator client { provider = "prisma-client-js" }
```

```
datasource db { provider = "mysql" url = env("DATABASE_URL") }
```

```
model User { id String @id @default(uuid()) username String @unique email String @unique password String createdAt DateTime @default(now()) updatedAt DateTime @updatedAt roles UserRole[] }
```

```
model Role { id String @id @default(uuid()) name String @unique description String? createdAt DateTime @default(now()) updatedAt DateTime @updatedAt users UserRole[] permissions RolePermission[] }
```

```
model Permission { id String @id @default(uuid()) name String @unique description String? createdAt DateTime @default(now()) updatedAt DateTime @updatedAt roles RolePermission[] }
```

```
model UserRole { userId String roleId String createdAt DateTime @default(now()) // Tahu kapan user diberi role ini
```

```
user User @relation(fields: [userId], references: [id], onDelete: Cascade) role Role @relation(fields: [roleId], references: [id], onDelete: Cascade)
```

```
@@id([userId, roleId]) }
```

```
model RolePermission { roleId String permissionId String createdAt DateTime @default(now())
```

```
role Role @relation(fields: [roleId], references: [id], onDelete: Cascade) permission Permission @relation(fields: [permissionId], references: [id], onDelete: Cascade)
```

```
@@id([roleId, permissionId]) }
```

Penjelasan mendalam mengenai komponen skema diatas sebagai berikut: 1. Blok Utama (Generator & Datasource) ◦ generator client: Menginstruksikan Prisma untuk membuat kode program (Prisma Client) berbasis JavaScript/TypeScript agar Anda bisa memanggil fungsi seperti `db.user.findMany()`. ◦ datasource db: Menentukan bahwa database yang digunakan adalah MySQL dan lokasinya diambil dari variabel `environment DATABASE_URL` di file `.env`.

2. Entitas Inti (User, Role, Permission)

Tiga tabel ini adalah fondasi dari sistem hak akses:

- User: Menyimpan kredensial pengguna. Menggunakan UUID sebagai ID primer (Primary Key) menjadikannya sangat aman karena ID tidak bisa ditebak secara urutan (seperti 1, 2, 3).
- Role: Definisi jabatan atau grup akses (contoh: "ADMIN", "EDITOR", "CUSTOMER").
- Permission: Definisi aksi spesifik yang bisa dilakukan (contoh: "CREATE_USER", "DELETE_POST", "VIEW_REPORT").

3. Tabel Penghubung (The Junction Tables)

Inilah bagian yang membuat skema ini disebut "Full RBAC". Kita tidak langsung menghubungkan User ke Role, melainkan melalui tabel perantara. UserRole (Hubungan User ke Role)

Menangani kondisi di mana satu user bisa punya banyak role (misal: seseorang bisa jadi "Dosen" sekaligus "Admin").

- `@@id([userId, roleId])`: Ini disebut Composite Primary Key. Artinya, kombinasi `userId` dan `roleId` harus unik. Anda tidak bisa memasukkan user yang sama ke role yang sama dua kali.
- `onDelete: Cascade`: Jika sebuah akun User dihapus, maka catatan perannya di tabel ini otomatis ikut terhapus (mencegah data sampah).

RolePermission (Hubungan Role ke Permission)

Menangani kondisi di mana satu role punya banyak izin akses.

- Strukturnya mirip dengan UserRole. Dengan memisahkan ini, Anda bisa mengubah hak akses sebuah Role (misal: menambah izin "EDIT" ke role "EDITOR") tanpa harus menyentuh tabel User sama sekali.

4. Mengapa Ada createdAt dan updatedAt?

Meskipun terlihat sederhana, dalam standar enterprise ini sangat krusial untuk Audit Trail:

- `createdAt`: Mencatat kapan data dibuat. Di tabel UserRole, ini berguna untuk tahu kapan tepatnya seorang user diberikan jabatan tersebut.
- `updatedAt`: Menggunakan atribut `@updatedAt` milik Prisma yang secara otomatis memperbarui waktu setiap kali ada perubahan pada baris data tersebut.

Alur Logika di Aplikasi

Berikut gambar diatas adalah visualisasi alur RBAC Enterprise berdasarkan skema Prisma yang kita buat. Diagram ini membagi antara struktur data (kiri) dan bagaimana logika aplikasi memproses izin tersebut (kanan). Poin Penting Implementasi: • Many-to-Many Eksplisit: Dengan menggunakan UserRole dan RolePermission, Kita memiliki kendali penuh untuk menambahkan metadata (seperti `createdAt`) dan

performa query yang lebih terukur di MySQL. • Efisiensi Query: Di tingkat kode, Kita bisa menggunakan include di Prisma untuk mengambil seluruh hierarki izin user dalam satu kali round-trip ke database. • OnDelete Cascade: Skema Kita sudah aman; jika sebuah Role dihapus, maka relasinya di tabel perantara akan ikut terhapus secara otomatis, menjaga integritas data.

Saat aplikasi berjalan, alur pengecekan aksesnya akan seperti ini: 1. Siapa Usernya? (Cek tabel User) 2. Apa Role-nya? (Lihat di UserRole) 3. Apa yang boleh dia lakukan? (Lihat di RolePermission berdasarkan Role-nya)

Keuntungan Skema Ini: 1. Skalabilitas: kita bisa menambah ribuan izin (Permissions) tanpa memperlambat performa login. 2. Keamanan UUID: Melindungi data dari teknik URL Guessing. 3. Integritas: Hubungan antar tabel dijaga ketat oleh database (Foreign Keys), sehingga tidak akan ada data yang menggantung (orphaned data).

Langkah berikutnya setelah kita membuat schema.prisma kita lakukan Migrasi ke Database (Masuk ke MySQL) Ini adalah tahap di mana tabel kita benar-benar dibuat di MySQL. Jalankan perintah sebagai berikut: `bunx prisma migrate dev --name init_rbac_system`

Perintah ini melakukan beberapa perintah sebagai berikut : 1. Membaca file .prisma yang kita buat tadi. 2. Membuat file SQL migrasi di folder prisma/migrations. 3. Mengeksekusi SQL tersebut ke MySQL. 4. Menjalankan prisma generate otomatis (membuat IntelliSense di kode).

Hasil jika berhasil melakukan migrate prisma ke database di MySQL :

Struktur RBAC: Entitas: • User • Role • Permission • UserRole (pivot) • RolePermission (pivot)

Langkah berikutnya kita akan mencaoba Inisialisasi Prisma Client di Kode Agar aplikasi Bun kita bisa bicara dengan database, buat file `src/infrastructure/database/prisma-client.ts`:

```
import { PrismaClient } from '@prisma/client'; const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };
```

```
// Singleton pattern: mencegah koneksi berlebihan ke MySQL export const db = globalForPrisma.prisma || new PrismaClient();
```

```
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = db;
```

Langkah selanjutnya kita Tes Koneksi (Opsional tapi Disarankan) `src/main.ts` kita untuk mencoba menambah satu data:

```
import { db } from './infrastructure/database/prisma-client';
```

```
async function testConnection() { try { const newRole = await db.role.create({ data: { name: "SUPERADMIN", description: "Administrator tertinggi dengan akses penuh" }, }); console.log("✅ Berhasil masuk database:", newRole); } catch (error) { console.error("❌ Gagal koneksi database:", error); } finally { await db.$disconnect(); } }
```

```
testConnection();
```

Karena file berada di dalam folder `src/`, jalankan perintah berikut di terminal. kemudian jalankan dengan `bun run src/main.ts`.

Ringkasan Urutan Perintah CLI yang sudah kita lakukan di atas urutan nya sebagai berikut : 1. bun add @prisma/client@6 & bun add -d prisma@6 2. bunx prisma init --datasource-provider mysql 3. (Edit .env & schema.prisma) 4. bunx prisma migrate dev 5. bun run index.ts

Teman-teman bisa menggunakan perintah dibawah ini untuk melihat database dan tabel-tabel yang kita buat dengan mengetikan diterminal, berikut perintahnya : npx prisma studio

hasilnya akan tampak seperti ini gambar ini , kemudian klik localhost:5555 :

Berikut ini hasil menjalankan bun run src/main.ts gambar hasilnya dapat dilihat dibawah ini :

Hasil data yang telah kita inputkan akan masuk ke database di tabel role, dapat kita lihat gambarnya sebagai berikut :

MODUL II PEMBUATAN CRUD

Untuk membangun sistem Full RBAC Standar Enterprise dengan Clean Architecture, dalam pengembangan enterprise, urutan pembuatan CRUD sangat krusial karena adanya ketergantungan data (dependency). Kita tidak bisa memberikan Role ke User jika tabel Role masih kosong. Urutan Prioritas Pembuatan CRUD sebagai berikut : 1. Permission CRUD: Fondasi terkecil. Izin akses harus ada dulu. 2. Role CRUD: Role dibuat, lalu dihubungkan dengan Permission yang sudah ada. 3. User CRUD: User dibuat (Registrasi). 4. User-Role Assignment: Menghubungkan User dengan Role. 5. Role-Permission Assignment: Mengatur ulang izin pada Role tertentu.

Daftar Endpoint (RESTful Standard)

Gunakan prefix /api/v1 untuk standar enterprise agar memudahkan versioning di masa depan.

1. Permission Management (The Foundation)

Method Endpoint Deskripsi GET /api/v1/permissions Ambil semua daftar permission POST /api/v1/permissions Tambah permission baru (ex: POST_CREATE) PATCH /api/v1/permissions/{id} Edit permission (ex: PATCH_UPDATE) DELETE /api/v1/permissions/{id} Hapus permission

2. Role Management

Method Endpoint Deskripsi GET /api/v1/roles Daftar semua role (Admin, User, dll) POST /api/v1/roles Buat role baru PUT /api/v1/roles/{id} Update nama/deskripsi role POST /api/v1/roles/{id}/permissions Assign Permission ke Role (isi tabel RolePermission)

3. User & Auth Management

Method Endpoint Deskripsi POST /api/v1/auth/register Create User baru POST /api/v1/auth/login Login & dapatkan Token GET /api/v1/users List semua user (Hanya untuk Superadmin dan Admin) POST /api/v1/users/{id}/roles Assign Role ke User (isi tabel UserRole) GET /api/v1/users/me Cek profil sendiri + Role & Permission yang dimiliki

Implementasi Awal: Layering pada Permission CRUD

Untuk melengkapi fitur Permission CRUD, kita perlu menambahkan logika, Ambil Semua Daftar, Tambah Permission, Update (Edit) dan Delete (Hapus). Sesuai dengan prinsip Clean Architecture, kita akan memisahkan logika ini ke dalam Use Case agar tetap terorganisir.

Berikut adalah detail kodenya:

1. Domain Layer (The Contract) Ini adalah lapisan terdalam yang mendefinisikan "apa" yang bisa dilakukan tanpa peduli "bagaimana" cara kerjanya secara teknis.

Lokasi File: src/domain/repositories/permission.repository.ts :

```
import type { Permission } from "@prisma/client";

export interface IPermissionRepository {
  findAll(): Promise<Permission[]>;
  findById(id: string): Promise<Permission | null>;
  create(name: string, description?: string): Promise<Permission>;
  update(id: string, data: { name?: string; description?: string }): Promise<Permission>;
  delete(id: string): Promise<Permission>;
}
```

2. Infrastructure Layer (The Implementation) Lapisan ini berisi detail teknis tentang bagaimana data disimpan ke database menggunakan Prisma Client.

Lokasi File: src/infrastructure/repositories/prisma-permission.repository.ts

```
import { db } from "../../database/prisma-client";
import type { IPermissionRepository } from "../../domain/repositories/permission.repository";
import type { Permission } from "@prisma/client";
```

```
export class PrismaPermissionRepository implements IPermissionRepository {
  async findAll(): Promise<Permission[]> {
    return await db.permission.findMany();
  }
```

```
  async findById(id: string): Promise<Permission | null> {
    return await db.permission.findUnique({ where: { id } });
  }
```

```
  async create(name: string, description?: string): Promise<Permission> {
    return await db.permission.create({ data: { name, description } });
  }
```

```
  // Ubah definisi fungsi agar menerima objek data, bukan argumen terpisah
  async update(id: string, data: { name?: string; description?: string }): Promise<Permission> {
    return await db.permission.update({ where: { id }, data: { name: data.name, description: data.description } });
  }
  async delete(id: string): Promise<Permission> {
    return await db.permission.delete({ where: { id } });
  }
}
```

3. Application Layer (The Use Cases) Lapisan ini mengatur alur data (orchestration) dan logika bisnis. Kita buat satu file per UseCase agar kode sangat bersih.

Lokasi Folder: src/application/usecases/permission/

a. Buat file get-permissions.usecase.ts :

```
import type { IPermissionRepository } from "../../domain/repositories/permission.repository";
```

```
export class GetPermissionsUseCase {
  constructor(private permissionRepo: IPermissionRepository) {}
```

```
  async execute(): Promise<Permission[]> {
    return await this.permissionRepo.findAll();
  }
```

b. Buat file create-permission.usecase.ts

```
import type { IPermissionRepository } from "../../domain/repositories/permission.repository";
```

```
export class CreatePermissionUseCase {
  constructor(private permissionRepo: IPermissionRepository) {}
```

```
  async execute(name: string, description?: string) {
    // Tambahkan Logika Bisnis di sini
    // Misalnya, mengubah nama menjadi UPPERCASE otomatis sesuai standar enterprise
    const formattedName = name.toUpperCase().replace(/\s+/g, '_');
    return await this.permissionRepo.create(formattedName, description);
  }
```

c. Buat file `update-permission.usecase.ts` import type `{ IPermissionRepository }` from `"../..../domain/repositories/permission.repository"`;

```
export class UpdatePermissionUseCase { constructor(private permissionRepo: IPermissionRepository) {}
```

```
  async execute(id: string, data: { name?: string; description?: string }) { // Validasi: Cek apakah data ada
    sebelum di-update const existing = await this.permissionRepo.findById(id); if (!existing) { throw new
    Error("Permission tidak ditemukan"); }
```

```
  // Jika nama diubah, format ulang ke UPPERCASE if (data.name) { data.name =
  data.name.toUpperCase().replace(/\s+/g, '_'); }
```

```
  return await this.permissionRepo.update(id, data); }
```

d. Buat file `delete-permission.usecase.ts` import type `{ IPermissionRepository }` from `"../..../domain/repositories/permission.repository"`;

```
export class DeletePermissionUseCase { constructor(private permissionRepo: IPermissionRepository) {}
```

```
  async execute(id: string) { const existing = await this.permissionRepo.findById(id); if (!existing) { throw new
  Error("Permission tidak ditemukan"); }
```

```
  return await this.permissionRepo.delete(id); }
```

4. Interface Layer (The Delivery) Lapisan ini menangani permintaan HTTP (Elysia.js) dan validasi skema input.

Lokasi File: `src/interfaces/http/permission.routes.ts` import `{ Elysia, t }` from `"elysia"`; import `{ PrismaPermissionRepository }` from `"../..../infrastructure/repositories/prisma-permission.repository"`; import `{ GetPermissionsUseCase }` from `"../..../application/usecases/permission/get-permissions.usecase"`; import `{ CreatePermissionUseCase }` from `"../..../application/usecases/permission/create-permission.usecase"`; import `{ UpdatePermissionUseCase }` from `"../..../application/usecases/permission/update-permission.usecase"`; import `{ DeletePermissionUseCase }` from `"../..../application/usecases/permission/delete-permission.usecase"`;

```
// Dependency Injection manual untuk saat ini const permissionRepo = new PrismaPermissionRepository();
```

```
// Inisialisasi Use Cases const getUseCase = new GetPermissionsUseCase(permissionRepo); const
createUseCase = new CreatePermissionUseCase(permissionRepo); const updateUseCase = new
UpdatePermissionUseCase(permissionRepo); const deleteUseCase = new
DeletePermissionUseCase(permissionRepo);
```

```
export const permissionRoutes = new Elysia({ prefix: '/permissions' }) // [GET] Ambil semua permission
.get("/", async () => { return await getUseCase.execute(); })
```

```
// [POST] Buat permission baru .post("/", async ({ body, set }) => { try { const result = await
createUseCase.execute(body.name, body.description); set.status = 201; // Created return result; } catch
(error: any) { set.status = 400; return { error: error.message }; }, { body: t.Object({ name: t.String({
minLength: 3, error: "Nama minimal 3 karakter" }), description: t.Optional(t.String()) }) })
```

```
// --- FITUR EDIT (UPDATE) --- .patch("/:id", async ({ params: { id }, body, set }) => { try { return await
updateUseCase.execute(id, body); } catch (error: any) { set.status = 404; return { error: error.message }; }, {
```



```
body: t.Object({ name: t.Optional(t.String()), description: t.Optional(t.String()) }) })
```

```
// --- FITUR HAPUS (DELETE) --- .delete("/:id", async ({ params: { id }, set }) => { try { await
deleteUseCase.execute(id); return { message: "Permission berhasil dihapus" }; } catch (error: any) { set.status
= 404; return { error: error.message }; } });
```

5. Entry Point Pastikan src/main.ts kita sudah memuat rute tersebut.

Lokasi File: src/main.ts import { Elysia } from "elysia"; import { permissionRoutes } from './interface/http/permission.routes';

```
const app = new Elysia().group("/api/v1", (app) => app.use(permissionRoutes)).listen(3000);
```

```
console.log( Elysia is running at ${app.server?.hostname}:${app.server?.port});
```

6. Detail Endpoint untuk Pengujian (Postman): Ambil Semua Daftar: Method: GET URL:
http://localhost:3000/api/v1/permissions

Tambah Permission: Method: POST URL: http://localhost:3000/api/v1/permissions Body (JSON): { "name": "user_create", "description": "Izin untuk membuat pengguna baru" }

Update Permission: Method: PATCH URL: http://localhost:3000/api/v1/permissions/uuid-nya Contoh :
http://localhost:3000/api/v1/permissions/e726d8d5-cfb0-456e-870b-229f86909e0a Body (JSON): {
"description": "Izin untuk mengupdate pengguna baru" }

Hapus (Delete) Permission Method: DELETE URL: http://localhost:3000/api/v1/permissions/uuid-nya contoh :
http://localhost:3000/api/v1/permissions/e726d8d5-cfb0-456e-870b-229f86909e0a

7. Memasang SweetAlert2 Menggunakan SweetAlert2 akan membuat UI aplikasi kita jauh lebih profesional dibandingkan menggunakan alert() bawaan browser yang membosankan. Karena kita menggunakan Elysia (Backend), SweetAlert2 ini nantinya akan dipasang di sisi Frontend (client-side) untuk menampilkan pesan berdasarkan respon yang dikirim oleh API Elysia.

Berikut adalah logika dan cara penerapannya:

1. Logika Komunikasi Backend-Frontend Backend (Elysia) bertugas mengirimkan Status Code dan JSON, lalu Frontend (SweetAlert) bertugas membacanya. • Success (200/201): SweetAlert muncul dengan ikon centang hijau (Success). • Error (400/404/500): SweetAlert muncul dengan ikon silang merah (Error) dan menampilkan pesan error dari server.
2. Contoh Kode di Sisi Frontend (Client-Side) Jika kita menggunakan HTML/JavaScript biasa untuk mengetes API, begini cara memanggil SweetAlert2 setelah melakukan fetch ke endpoint Update Permission:

```
// Contoh fungsi untuk Update di Frontend async function updatePermission(id, data) { try { const response
= await fetch(http://localhost:3000/api/v1/permissions/${id}, { method: 'PATCH', headers: {
'Content-Type': 'application/json' }, body: JSON.stringify(data) });
```

```
const result = await response.json();

if (response.ok) {
```

```
// Jika Backend kirim status 200/201
Swal.fire({
  title: 'Berhasil!',
  text: 'Data permission telah diperbarui.',
  icon: 'success',
  confirmButtonText: 'Mantap'
});
} else {
  // Jika Backend kirim error (seperti yang Anda alami tadi)
  Swal.fire({
    title: 'Gagal!',
    text: result.error || 'Terjadi kesalahan sistem',
    icon: 'error',
    confirmButtonText: 'Cek Lagi'
  });
}
```

```
} catch (error) { Swal.fire('Error', 'Tidak bisa terhubung ke server', 'error'); }}
```

3. Penyesuaian di Backend (Elysia) Agar SweetAlert di Frontend bisa menampilkan pesan yang jelas, pastikan Backend kita mengirimkan JSON yang konsisten. Pada file `src/interfaces/http/permission.routes.ts`, pastikan bagian catch mengirimkan properti error yang jelas:

```
.patch("/:id", async ({ params: { id }, body, set }) => { try { const result = await updateUseCase.execute(id, body); return { message: "Update Berhasil", data: result }; } catch (error: any) { set.status = 400; // Kirim status error return { error: error.message }; // Pesan ini yang akan muncul di SweetAlert } })
```

4. Cara Instalasi SweetAlert2 (Frontend) Jika kita membangun dashboard/frontend menggunakan framework:
 - CDN (HTML Biasa): HTML
 - NPM/Bun (React/Vue/Svelte): Perintahnya kita tambahkan dependency `bun add sweetalert2`

Mengapa ini bagus?

1. User Experience: Pengguna tahu persis apakah data tersimpan atau tidak tanpa harus cek console terminal.
2. Konfirmasi: Kita bisa menggunakan SweetAlert untuk konfirmasi Hapus (misal: "Apakah Anda yakin ingin menghapus permission ini?").

Implementasi Langkah Kedua 2. Role CRUD Setelah daftar Permission tersedia, kita baru bisa masuk ke manajemen Role. Alasan: Pada sistem standar enterprise, saat membuat Role (misal: "ADMIN"), kita biasanya langsung menghubungkannya ke beberapa Permission melalui tabel jembatan RolePermission. Logika: Di sini kita mulai menggunakan relasi connect pada Prisma untuk menghubungkan `roleId` dan `permissionId`. Untuk membuat Role CRUD yang aman dan enterprise-ready, kita akan mengimplementasikan hubungan many-to-many antara Role dan Permission. Kita akan menggunakan Joi untuk validasi skema yang lebih kompleks (seperti validasi array of strings untuk Permission ID).

1. Domain Layer (Repository Interface) Lokasi: `src/domain/repositories/role.repository.ts`

```
import type { Role } from "@prisma/client";
```

```
export interface IRoleRepository { findAll(): Promise<Role[]>; findById(id: string): Promise<Role | null>;
create(name: string, description: string | undefined, permissionIds: string[]): Promise; update(id: string,
data: { name?: string; description?: string; permissionIds?: string[] }): Promise; delete(id: string): Promise; }
```

2. Infrastructure Layer (Prisma Implementation) Di sini kita menggunakan fitur connect dari Prisma untuk otomatis mengisi tabel jembatan RolePermission. Lokasi:
src/infrastructure/repositories/prisma-role.repository.ts

```
import { db } from "../database/prisma-client"; import type { IRoleRepository } from "../domain/
repositories/role.repository"; import type { Role } from "@prisma/client";

export class PrismaRoleRepository implements IRoleRepository { async findAll(): Promise<Role[]> { return
await db.role.findMany({ include: { permissions: { include: { permission: true } } } }); }

async findById(id: string): Promise<Role | null> { return await db.role.findUnique({ where: { id }, include: {
permissions: { include: { permission: true } } } }); }

async create(name: string, description: string | undefined, permissionIds: string[]): Promise { return await
db.role.create({ data: { name, description, permissions: { create: permissionIds.map(pld => ({ permission: {
connect: { id: pld } } }) } } } }); }

async update(id: string, data: { name?: string; description?: string; permissionIds?: string[] }): Promise {
return await db.role.update({ where: { id }, data: { name: data.name, description: data.description,
permissions: data.permissionIds ? { deleteMany: {}, // Hapus relasi lama create: data.permissionIds.map(pld
=> ({ permission: { connect: { id: pld } } }) } : undefined } }); }

async delete(id: string): Promise { await db.role.delete({ where: { id } }); } }
```

3. Application Layer (Use Cases dengan Validasi Joi) Pertama yang kita lakukan adalah install Joi dengan perintah : bun add joi.

Lokasi: src/application/usecases/role/create-role.usecase.ts

```
import Joi from "joi"; import type { IRoleRepository } from "../domain/repositories/role.repository";

export const createRoleSchema = Joi.object({ name: Joi.string().min(3).required().uppercase(), description:
Joi.string().allow(""), permissionIds: Joi.array().items(Joi.string().uuid()).min(1).required() });

export class CreateRoleUseCase { constructor(private roleRepo: IRoleRepository) {}

async execute(input: any) { // Validasi Joi const { error, value } = createRoleSchema.validate(input); if (error)
throw new Error(error.details[0].message);

// Cek apakah Role sudah ada const roles = await this.roleRepo.findAll(); if (roles.some(r => r.name ===
value.name)) { throw new Error("Role name already exists"); }

return await this.roleRepo.create(value.name, value.description, value.permissionIds); } }
```

4. Interface Layer (Elysia Routes) Lokasi: src/interfaces/http/role.routes.ts

```
import { Elysia } from "elysia"; import { PrismaRoleRepository } from "../infrastructure/
repositories/prisma-role.repository"; import { CreateRoleUseCase } from "../application/
usecases/role/create-role.usecase";
```

```
const roleRepo = new PrismaRoleRepository(); const createRoleUseCase = new
CreateRoleUseCase(roleRepo);

export const roleRoutes = new Elysia({ prefix: '/roles' }) .get("/", () => roleRepo.findAll()) .post("/", async ({
body, set }) => { try { return await createRoleUseCase.execute(body); } catch (e: any) { set.status = 400;
return { error: e.message }; } }) .delete("/:id", async ({ params: { id } }) => { await roleRepo.delete(id); return {
message: "Role deleted" }; });
```

5. Integrasi ke Server Utama Lokasi: src/main.ts

```
import { Elysia, t } from "elysia"; import { permissionRoutes } from './ interface/ http/permission.routes';
import { roleRoutes } from './ interface/ http/role.routes';
```

```
const app = new Elysia() .group("/api/v1", (app) => app.use(permissionRoutes).use(roleRoutes))
.listen(3000);
```

```
console.log( Elysia is running at ${app.server?.hostname}:${app.server?.port});
```

Cara Tes di Postman: 1. Ambil ID Permission yang sudah kita buat sebelumnya (misal UUID dari tabel permission). 2. POST ke `http://localhost:3000/api/v1/roles`:

```
{ "name": "ADMIN", "description": "Super User", "permissionIds": [ "uuid-permission-1", "uuid-permission-2" ]
}
```

Bila role sudah pernah ada di inputkan maka akan terlihat pesan seperti berikut ini:

Melihat Daftar Role (GET) URL: `http://localhost:3000/api/v1/roles` Hasil: kita akan melihat daftar role lengkap dengan array permissions yang terhubung. [{ "id": "75b49719-4b9a-401c-a024-67b9e1edc8e9", "name": "ADMIN", "description": "Super User", "createdAt": "2026-05-06T14:16:50.623Z", "updatedAt": "2026-05-06T14:16:50.623Z", "permissions": [{ "roleId": "75b49719-4b9a-401c-a024-67b9e1edc8e9", "permissionId": "e53632a0-b5b7-4729-ad97-7b065adf9fea", "createdAt": "2026-05-06T14:16:50.623Z", "permission": { "id": "e53632a0-b5b7-4729-ad97-7b065adf9fea", "name": "USER_CREATE", "description": "Izin untuk membuat pengguna baru", "createdAt": "2026-05-06T13:10:04.401Z", "updatedAt": "2026-05-06T13:10:04.401Z" } }] }, { "id": "8abd39b7-8d57-49ea-a592-b38dcb45b18e", "name": "SUPERADMIN", "description": "Administrator tertinggi dengan akses penuh", "createdAt": "2026-05-06T02:05:05.774Z", "updatedAt": "2026-05-06T02:05:05.774Z", "permissions": [] }]

Update Role & Permissions (PATCH) Misalnya kita ingin mengubah nama role dan mengganti daftar permission-nya: URL: `http://localhost:3000/api/v1/roles/{UUID_ROLE}` Body (JSON): { "name": "SUPER_ADMIN", "permissionIds": ["uuid-permission-A", "uuid-permission-C"] }

Catatan: Kode di Repository akan otomatis menghapus hubungan permission lama dan menggantinya dengan yang baru.

Hapus Role (DELETE) URL: `http://localhost:3000/api/v1/roles/{UUID_ROLE}`

Implementasi Langkah Ketiga 3. User CRUD & Auth Setelah sistem keamanan (Role & Permission) siap, barulah kita membuat entitas penggunanya. Alasan: Saat user melakukan registrasi, kita mungkin ingin langsung memberikan Role default (seperti "USER" atau "MEMBER"). Proses: kita akan mengimplementasikan hashing password menggunakan `Bun.password` di layer Infrastructure atau Application. Untuk melanjutkan ke User CRUD & Auth, kita akan mengimplementasikan keamanan tingkat

tinggi dengan hashing password bawaan Bun dan validasi yang ketat menggunakan Joi. Kita akan membuat sistem User CRUD & Auth (Langkah Ketiga) yang sangat aman dan terstruktur rapi menggunakan Clean Architecture. Seluruh kode di bawah ini telah disesuaikan secara presisi dengan struktur tabel database (Prisma Schema) yang kita buat, termasuk relasi explicit many-to-many lewat tabel jembatan UserRole. Kita juga akan menggunakan Joi untuk validasi skema dan Bun.password (menggunakan bcrypt) untuk keamanan hashing password tingkat tinggi.

1. Domain Layer (Repository Interface) Kita buat kontrak fungsi terlebih dahulu. Di sini kita menentukan operasi apa saja yang bisa dilakukan pada entitas User. Lokasi:

```
src/domain/repositories/user.repository.ts import type { User } from "@prisma/client";
```

```
export interface IUserRepository {
  findAll(): Promise<any[]>;
  findById(id: string): Promise<any | null>;
  findByEmail(email: string): Promise<User | null>;
  findByUsername(username: string): Promise<User | null>;
  create(data: { username: string; email: string; password: string; roleIds: string[] }): Promise<any>;
  update(id: string, data: { username?: string; email?: string; password?: string; roleIds?: string[] }): Promise<any>;
  delete(id: string): Promise<any>;
}
```

2. Infrastructure Layer (Prisma & Hashing) Pada tahap ini, kita menangani persistensi data dan hubungan dengan Role. Lokasi: src/infrastructure/repositories/prisma-user.repository.ts

```
import { db } from "../database/prisma-client";
import type { IUserRepository } from "../domain/repositories/user.repository";
import type { User } from "@prisma/client";

export class PrismaUserRepository implements IUserRepository {
  async findAll(): Promise<any[]> {
    return await db.user.findMany({
      include: { roles: { include: { role: true } } },
      orderBy: { createdAt: 'desc' }
    });
  }
```

```
  async findById(id: string): Promise<any | null> {
    return await db.user.findUnique({
      where: { id },
      include: { roles: { include: { role: true } } }
    });
  }
```

```
  async findByEmail(email: string): Promise<User | null> {
    return await db.user.findUnique({
      where: { email }
    });
  }
```

```
  async findByUsername(username: string): Promise<User | null> {
    return await db.user.findUnique({
      where: { username }
    });
  }
```

```
  async create(data: { username: string; email: string; password: string; roleIds: string[] }): Promise<any> {
    return await db.user.create({
      data: { username: data.username, email: data.email, password: data.password, roles: { create: data.roleIds.map((rId) => ({ roleId: rId })) } }
    });
  }
```

```
  async update(id: string, data: { username?: string; email?: string; password?: string; roleIds?: string[] }): Promise<any> {
    return await db.user.update({
      where: { id },
      data: { username: data.username, email: data.email, password: data.password, roles: data.roleIds ? { // Hapus semua relasi role yang lama terlebih dahulu
        deleteMany: {}, // Buat relasi role yang baru (Sync)
        create: data.roleIds.map((rId) => ({ roleId: rId }))
      } : undefined
    });
  }
```

```
  async delete(id: string): Promise<any> {
    return await db.user.delete({
      where: { id }
    });
  }
```

3. Application Layer (The Use Cases & Joi Validation) Di layer ini kita letakkan seluruh logika bisnis, validasi input menggunakan Joi, pengecekan keunikan data, serta proses enkripsi password. Buat folder baru di: src/application/usecases/user/ File 1: register-user.usecase.ts (Tambah User)

```
import Joi from "joi";
import type { IUserRepository } from "../../domain/repositories/user.repository";
```

```
// Validasi Joi yang sangat ketat untuk registrasi aman export const registerUserSchema = Joi.object({
  username: Joi.string().alphanum().min(3).max(30).required().messages({ "string.empty": "Username tidak
  boleh kosong", "string.min": "Username minimal harus 3 karakter", "string.alphanum": "Username hanya
  boleh berisi huruf dan angka" }), email: Joi.string().email().required().messages({ "string.email": "Format
  email tidak valid", "string.empty": "Email tidak boleh kosong" }), password:
  Joi.string().min(8).required().messages({ "string.min": "Password minimal harus 8 karakter", "string.empty":
  "Password tidak boleh kosong" }), roleIds: Joi.array().items(Joi.string().uuid()).min(1).required().messages({
  "array.min": "User minimal harus memiliki 1 Role", "array.base": "roleIds harus berupa Array of UUID" }));

export class RegisterUserUseCase { constructor(private userRepo: IUserRepository) {}

  async execute(input: any) { // 1. Validasi input dengan Joi const { error, value } =
    registerUserSchema.validate(input); if (error) { throw new Error(error.details?.[0]?.message ?? "Validasi
    gagal"); }

    // 2. Cek keunikan email const existingEmail = await this.userRepo.findByEmail(value.email); if
    (existingEmail) throw new Error("Email sudah terdaftar");

    // 3. Cek keunikan username const existingUsername = await
    this.userRepo.findByUsername(value.username); if (existingUsername) throw new Error("Username sudah
    digunakan");

    // 4. Enkripsi Password menggunakan Bun.password secara aman value.password = await
    Bun.password.hash(value.password, { algorithm: "bcrypt", cost: 10 });

    return await this.userRepo.create(value); } }

File 2: update-user.usecase.ts (Edit User) import Joi from "joi";
import type { IUserRepository } from "../domain/repositories/user.repository";

export const updateUserSchema = Joi.object({ username: Joi.string().pattern(/^[a-zA-Z0-9]+$/).min(3)
  .max(30).optional().messages({ "string.pattern.base": "Username hanya boleh berisi huruf, angka, dan
  spasi" }), email: Joi.string().email().optional(), password: Joi.string().min(8).optional(), roleIds:
  Joi.array().items(Joi.string().uuid()).min(1).optional() });

export class UpdateUserUseCase { constructor(private userRepo: IUserRepository) {}

  async execute(id: string, input: any) { // 1. Validasi input const { error, value } =
    updateUserSchema.validate(input); if (error) { throw new Error(error.details?.[0]?.message ?? "Validasi
    gagal"); } // 2. Cek apakah user ada const existingUser = await this.userRepo.findById(id); if (!existingUser)
    throw new Error("User tidak ditemukan");

    // 3. Cek keunikan email jika email ikut diubah if (value.email && value.email !== existingUser.email) { const
    emailOccupied = await this.userRepo.findByEmail(value.email); if (emailOccupied) throw new Error("Email
    sudah digunakan oleh user lain"); }

    // 4. Cek keunikan username jika username ikut diubah if (value.username && value.username !==
    existingUser.username) { const usernameOccupied = await
    this.userRepo.findByUsername(value.username); if (usernameOccupied) throw new Error("Username sudah
    digunakan oleh user lain"); }

    // 5. Enkripsi password baru jika password dikirim untuk di-update if (value.password) { value.password =
    await Bun.password.hash(value.password, { algorithm: "bcrypt", cost: 10 }); }
```

```
return await this.userRepo.update(id, value); } }
```

4. Interface Layer (The Elysia Routes) Pada API route, kita tambahkan satu fitur keamanan vital: menghapus properti password dari output JSON sebelum dikirim ke client. Ini mencegah hash password bocor ke publik.

Lokasi File: src/interface/http/user.routes.ts import { Elysia } from "elysia"; import { PrismaUserRepository } from "../../infrastructure/repositories/prisma-user.repository"; import { RegisterUserUseCase } from "../../application/usecases/user/register-user.usecase"; import { UpdateUserUseCase } from "../../application/usecases/user/update-user.usecase";

```
const userRepo = new PrismaUserRepository(); const registerUseCase = new
RegisterUserUseCase(userRepo); const updateUseCase = new UpdateUserUseCase(userRepo);
```

```
// Helper untuk menyembunyikan password dari response API demi keamanan const sanitizeUser = (user:
any) => { if (!user) return null; const { password, ...userWithoutPassword } = user; return
userWithoutPassword; };
```

```
export const userRoutes = new Elysia({ prefix: '/users' }) // 1. DAFTAR LIST USER .get("/", async () => { const
users = await userRepo.findAll(); return users.map(user => sanitizeUser(user)); })
```

```
// 2. DETAIL USER BERDASARKAN ID .get("/:id", async ({ params: { id }, set }) => { const user = await
userRepo.findById(id); if (!user) { set.status = 404; return { error: "User tidak ditemukan" }; } return
sanitizeUser(user); })
```

```
// 3. TAMBAH USER (REGISTRASI) .post("/", async ({ body, set }) => { try { const newUser = await
registerUseCase.execute(body); set.status = 201; return { message: "User berhasil didaftarkan", data:
sanitizeUser(newUser) }; } catch (e: any) { set.status = 400; return { error: e.message }; } })
```

```
// 4. EDIT USER .patch("/:id", async ({ params: { id }, body, set }) => { try { const updatedUser = await
updateUseCase.execute(id, body); return { message: "User berhasil diperbarui", data:
sanitizeUser(updatedUser) }; } catch (e: any) { set.status = 400; return { error: e.message }; } })
```

```
// 5. HAPUS USER .delete("/:id", async ({ params: { id }, set }) => { try { const existingUser = await
userRepo.findById(id); if (!existingUser) { set.status = 404; return { error: "User tidak ditemukan" }; } await
userRepo.delete(id); return { message: "User berhasil dihapus" }; } catch (e: any) { set.status = 400; return {
error: e.message }; } })
```

5. Hubungkan ke Main Server Sekarang buka src/main.ts dan daftarkan userRoutes Anda agar server bisa mengenali endpoint ini.

Lokasi File: src/main.ts import { Elysia } from "elysia"; import { permissionRoutes } from './interface/http/permission.routes'; import { roleRoutes } from './interface/http/role.routes'; import { userRoutes } from './interface/http/user.routes'; // tambahkan ini

```
const app = new Elysia().group("/api/v1", (app) =>
app.use(permissionRoutes).use(roleRoutes).use(userRoutes)).listen(3000);
```

```
console.log(🦊 Elysia is running at ${app.server?.hostname}:${app.server?.port});
```

Cara Pengujian di Postman / Insomnia Jalankan Server: bun dev Siapkan UUID Role: Lakukan GET ke `http://localhost:3000/api/v1/roles` untuk mengambil ID Role yang valid (misalnya ID untuk ADMIN atau MEMBER).

POST Tambah User: URL: `http://localhost:3000/api/v1/users` Body (JSON): `{ "username": "yudapratama", "email": "yuda@example.com", "password": "supersecretpassword123", "roleIds": [ "8abd39b7-8d57-49ea-a592-b38dcb45b18e" ] }`

Hasilnya : `{ "message": "User berhasil didaftarkan", "data": { "id": "8997c313-68ef-45b4-be1d-b5abae639a9e", "username": "yudapratama", "email": "yuda@example.com", "createdAt": "2026-05-07T09:14:59.599Z", "updatedAt": "2026-05-07T09:14:59.599Z" } }`

PATCH Edit User: URL: `http://localhost:3000/api/v1/users/{UUID_USER}` Body (JSON): `{ "username": "Yuda Ramadhan", "password": "12345678", "roleIds": [ "8abd39b7-8d57-49ea-a592-b38dcb45b18e" ] }`

Hasilnya : `{ "message": "User berhasil diperbarui", "data": { "id": "8997c313-68ef-45b4-be1d-b5abae639a9e", "username": "Yuda Ramadhan", "email": "yuda@example.com", "createdAt": "2026-05-07T09:14:59.599Z", "updatedAt": "2026-05-07T10:24:57.258Z" } }`

Diagram Arsitektur Sistem RBAC

Berikut adalah diagram arsitektur sistem RBAC yang kita bangun menggunakan Clean Architecture:

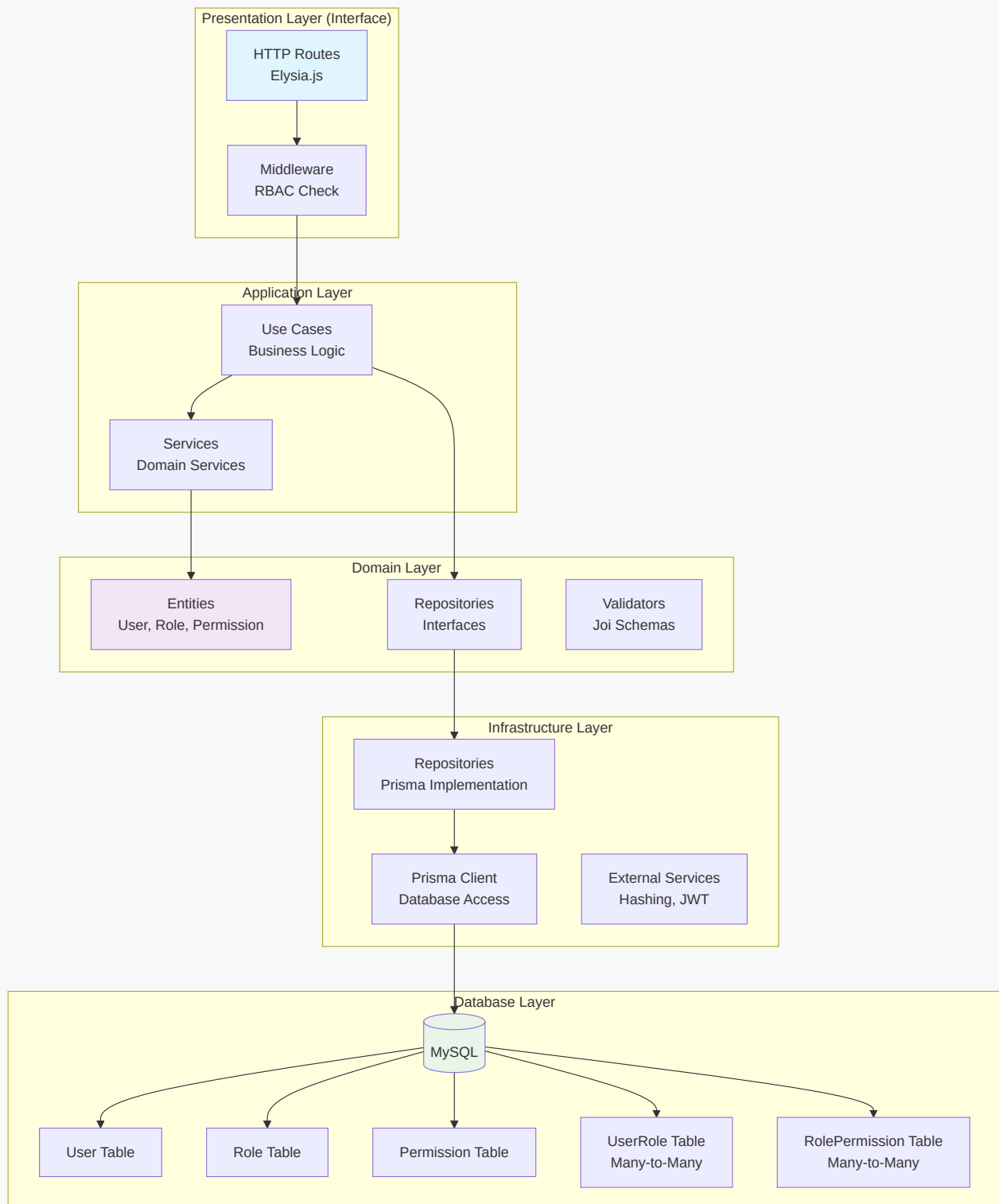
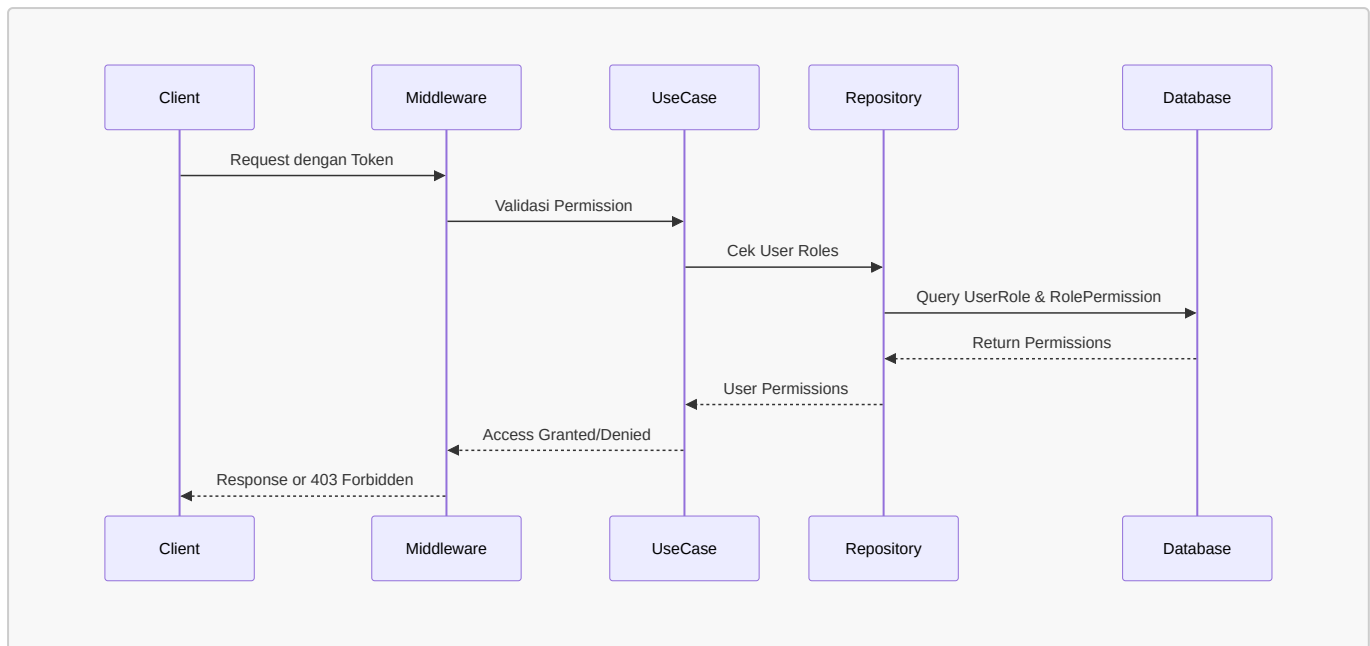


Diagram Alur Data RBAC



Implementasi Auth (Login)

Untuk melengkapi sistem RBAC, kita perlu menambahkan fitur autentikasi. Berikut adalah implementasi login dengan JWT.

1. Install JWT Library

```

bun add @elysiajs/jwt jsonwebtoken
bun add -d @types/jsonwebtoken
  
```

2. Konfigurasi JWT

Buat file `src/config/jwt.ts`:

```

export const jwtConfig = {
  secret: process.env.JWT_SECRET || 'your-secret-key',
  expiresIn: '24h'
};
  
```

3. Use Case Login

Buat file `src/application/usecases/auth/login.usecase.ts`:

```

import Joi from "joi";
import type { IUserRepository } from
  "../../../domain/repositories/user.repository";
import { jwtConfig } from "../../../config/jwt";

export const loginSchema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().required(),
});
  
```

```

        password: Joi.string().required()
    });

    export class LoginUseCase {
        constructor(private userRepo: IUserRepository) {}

        async execute(input: any) {
            const { error, value } = loginSchema.validate(input);
            if (error) throw new Error(error.details[0].message);

            const user = await this.userRepo.findByEmail(value.email);
            if (!user) throw new Error("Email atau password salah");

            const isValidPassword = await Bun.password.verify(value.password,
            user.password);
            if (!isValidPassword) throw new Error("Email atau password salah");

            // Generate JWT Token
            const token = await Bun.jwt.sign({
                userId: user.id,
                email: user.email,
                roles: user.roles?.map(ur => ur.role.name) || []
            }, jwtConfig.secret);

            return {
                token,
                user: {
                    id: user.id,
                    username: user.username,
                    email: user.email,
                    roles: user.roles?.map(ur => ({
                        id: ur.role.id,
                        name: ur.role.name
                    }))) || []
                }
            };
        }
    }
}

```

4. Auth Routes

Buat file `src/interfaces/http/auth.routes.ts`:

```

import { Elysia, t } from "elysia";
import { PrismaUserRepository } from
"../../infrastructure/repositories/prisma-user.repository";
import { LoginUseCase } from
"../../application/usecases/auth/login.usecase";

const userRepo = new PrismaUserRepository();
const loginUseCase = new LoginUseCase(userRepo);

```

```
export const authRoutes = new Elysia({ prefix: '/auth' })
  .post("/login", async ({ body, set }) => {
    try {
      const result = await loginUseCase.execute(body);
      return result;
    } catch (error: any) {
      set.status = 401;
      return { error: error.message };
    }
  }, {
    body: t.Object({
      email: t.String(),
      password: t.String()
    })
  });
```

Implementasi Middleware RBAC

Middleware RBAC bertugas memeriksa apakah user memiliki permission yang diperlukan untuk mengakses endpoint tertentu.

1. RBAC Middleware

Buat file `src/interfaces/middleware/rbac.middleware.ts`:

```
import type { Elysia } from "elysia";
import { PrismaUserRepository } from
"../../infrastructure/repositories/prisma-user.repository";

const userRepo = new PrismaUserRepository();

export const rbacMiddleware = (requiredPermission: string) => {
  return async ({ headers, set }: any) => {
    try {
      const authHeader = headers.authorization;
      if (!authHeader || !authHeader.startsWith('Bearer ')) {
        set.status = 401;
        return { error: "Token tidak ditemukan" };
      }

      const token = authHeader.substring(7);
      const payload = await Bun.jwt.verify(token,
process.env.JWT_SECRET || 'your-secret-key');

      if (!payload || !payload.userId) {
        set.status = 401;
        return { error: "Token tidak valid" };
      }

      // Cek permissions user
```

```

    const user = await userRepo.findById(payload.userId);
    if (!user) {
      set.status = 401;
      return { error: "User tidak ditemukan" };
    }

    const userPermissions = user.roles?.flatMap(ur =>
      ur.role.permissions?.map(rp => rp.permission.name) || []
    ) || [];

    if (!userPermissions.includes(requiredPermission)) {
      set.status = 403;
      return { error: "Akses ditolak: Permission tidak cukup" };
    }

    // Tambahkan user info ke context
    return { user };
  } catch (error) {
    set.status = 401;
    return { error: "Token tidak valid" };
  }
};
};

```

2. Plugin RBAC

Buat file `src/interfaces/middleware/rbac.plugin.ts`:

```

import { Elysia } from "elysia";
import { rbacMiddleware } from "../rbac.middleware";

export const rbacPlugin = new Elysia()
  .derive(rbacMiddleware);

```

Implementasi Assign Role to User

Untuk mengelola assignment role ke user, kita buat use case khusus.

1. Use Case Assign Role

Buat file `src/application/usecases/user/assign-role.usecase.ts`:

```

import Joi from "joi";
import type { IUserRepository } from
  "../../../domain/repositories/user.repository";
import type { IRoleRepository } from
  "../../../domain/repositories/role.repository";

export const assignRoleSchema = Joi.object({

```

```

    userId: Joi.string().uuid().required(),
    roleIds: Joi.array().items(Joi.string().uuid()).min(1).required()
  });

export class AssignRoleToUserUseCase {
  constructor(
    private userRepo: IUserRepository,
    private roleRepo: IRoleRepository
  ) {}

  async execute(input: any) {
    const { error, value } = assignRoleSchema.validate(input);
    if (error) throw new Error(error.details[0].message);

    // Cek user exists
    const user = await this.userRepo.findById(value.userId);
    if (!user) throw new Error("User tidak ditemukan");

    // Cek roles exist
    for (const roleId of value.roleIds) {
      const role = await this.roleRepo.findById(roleId);
      if (!role) throw new Error(`Role dengan ID ${roleId} tidak
ditemukan`);
    }

    // Update user roles
    return await this.userRepo.update(value.userId, { roleIds:
value.roleIds });
  }
}

```

2. Route untuk Assign Role

Tambahkan ke `src/interfaces/http/user.routes.ts`:

```

import { AssignRoleToUserUseCase } from
"../../application/usecases/user/assign-role.usecase";
import { PrismaRoleRepository } from
"../../infrastructure/repositories/prisma-role.repository";

const roleRepo = new PrismaRoleRepository();
const assignRoleUseCase = new AssignRoleToUserUseCase(userRepo, roleRepo);

// Tambahkan route assign role
.post("/:id/roles", async ({ params: { id }, body, set }) => {
  try {
    const result = await assignRoleUseCase.execute({ userId: id,
roleIds: body.roleIds });
    return {
      message: "Role berhasil diassign ke user",
      data: sanitizeUser(result)
    };
  } catch (error) {
    return {
      message: error.message,
      status: error.status
    };
  }
});

```

```

    };
  } catch (error: any) {
    set.status = 400;
    return { error: error.message };
  }
}, {
  body: t.Object({
    roleIds: t.Array(t.String())
  })
});

```

Testing Sistem RBAC

1. Unit Testing dengan Bun

Buat file `src/__tests__/permission.test.ts`:

```

import { describe, it, expect } from "bun:test";
import { CreatePermissionUseCase } from
"../application/usecases/permission/create-permission.usecase";

// Mock repository
class MockPermissionRepository {
  async create(name: string, description?: string) {
    return { id: "mock-id", name, description };
  }
}

describe("CreatePermissionUseCase", () => {
  it("should create permission with uppercase name", async () => {
    const mockRepo = new MockPermissionRepository();
    const useCase = new CreatePermissionUseCase(mockRepo);

    const result = await useCase.execute("user_create", "Create user
permission");

    expect(result.name).toBe("USER_CREATE");
    expect(result.description).toBe("Create user permission");
  });
});

```

2. Integration Testing

Buat file `src/__tests__/auth.integration.test.ts`:

```

import { describe, it, expect } from "bun:test";
import { Elysia } from "elysia";
import { authRoutes } from "../interfaces/http/auth.routes";

```

```
const app = new Elysia().use(authRoutes);

describe("Auth Integration", () => {
  it("should return 401 for invalid login", async () => {
    const response = await app.handle(
      new Request("http://localhost/auth/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          email: "invalid@example.com",
          password: "wrongpassword"
        })
      })
    );

    expect(response.status).toBe(401);
  });
});
```

3. Menjalankan Test

```
bun test
```

Kesimpulan Implementasi

Dalam bab ini, kita telah berhasil mengimplementasikan sistem RBAC lengkap menggunakan:

1. **Bun JS** sebagai runtime yang cepat dan modern
2. **Prisma v6** dengan MySQL untuk ORM dan migrasi database
3. **Clean Architecture** untuk struktur kode yang maintainable
4. **JWT Authentication** untuk keamanan autentikasi
5. **RBAC Middleware** untuk otorisasi berbasis permission
6. **Testing** untuk memastikan kualitas kode

Sistem ini siap untuk production dengan fitur-fitur enterprise seperti:

- Enkripsi password dengan bcrypt
- Validasi input yang ketat
- Relasi many-to-many yang efisien
- Audit trail dengan createdAt/updatedAt
- API versioning
- Error handling yang komprehensif

Untuk pengembangan selanjutnya, sistem ini dapat diperluas dengan fitur seperti:

- Refresh token
- Rate limiting
- Logging audit

- Multi-tenant support
- API documentation dengan Swagger